**Università degli Studi dell'Aquila**
**Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica**

---

**Software Engineering for the thr Internet of Things**

# IoT Intelligent Predictive Maintenace Project

**Professore:**
Prof. Davide Di Ruscio

**Studenti:**
Alessio Torrieri, Leonardo Pinterpe, Matteo Paolino

---

**Anno Accademico 2025-26**

# Indice

# Elenco delle figure

Capitolo 1

# Project proposal: IIoT system for predictive maintenance

## 1.1 Abstract

The project aims to develop a scalable, containerised Industrial Internet of Things (IIoT) infrastructure for real-time monitoring and predictive maintenance of industrial engines. The idea is to create a practical application that is able to anticipate critical failures. The proposed system covers and manages the entire data lifecycle: from telemetric generation via key sensors, to ingestion via communication protocol (like MQTT), logical processing with a proactive alert system and historical visualization on analytical dashboards showing the health status of assets. The architecture is designed to ensure interoperability and resilience, leveraging a Docker-based technology stack.

## 1.2 Application scenario: IIoT and Predictive Maintenance

We are moving into the realm of Industrial IoT. In the context of Industry 4.0, the sudden failure of a critical asset (e.g., a primary production line motor) results in high costs due to unplanned downtime. The traditional 'Fail-and-Fix' approach is

now obsolete. The aim of this project is to enable Condition-Based Maintenance (CBM) and lay the foundations for Predictive Maintenance. By constantly monitoring the vital parameters of machines, it is possible to detect the 'weak signals' of degradation (e.g. increased vibrations or temperature) long before catastrophic failure occurs, allowing for targeted and scheduled interventions.

## 1.3 Proposed Logical Architecture Details

The system should be designed according to modern microservices and containerization paradigms, ensuring scalability, isolation and portability on any infrastructure (Edge or Cloud). The data flow is structured in four logical phases:

**Data Ingestion & Simulation:** generation of industrial telemetry. The system simulates the physical behavior of complex machinery, replicating the vibrational, thermal and electrical signatures typical of different stages of operation and degradation.

**Message Brokerage:** Use of industry-standard M2M (Machine-to-Machine) protocols for lightweight, asynchronous data transport, ensuring decoupling between field sensors and analysis systems.

**Processing & Storage:** A Time-Series processing engine acquires data streams, applying validation logic and historising metrics in databases optimised for high-frequency time series.

**Visualization & Decision Support:** Interactive dashboards transform raw data into actionable information, allowing immediate comparison between operating values and regulatory safety thresholds (e.g., ISO standards for vibrations).

## 1.4 Project Objectives

**High Fidelity Simulation:** Creation of a virtual environment that replicates the physical behavior of industrial engines, including statistical variance and physical vibration models (RMS).

**Microservices Architecture:** Implementation of a modular system isolated via Docker Container to ensure portability and ease of deployment.

**Real-time Monitoring & Alerting:** Development of control logic capable of instantly assessing when critical thresholds (compliant with ISO/IEC standards) are exceeded and notifying operators.

**Historical Analysis:** Historization of time series on an optimized database and advanced visualization for trend analysis.

## 1.5 Key Monitoring Parameters

To validate the diagnostic effectiveness of the platform, the system will focus on three physical parameters:

- **Mechanical Vibrations:** An early indicator of imbalances or bearing defects.
- **Operating Temperature:** A symptom of friction, overload or cooling problems.
- **Electrical Absorption:** Metric for assessing motor stress and energy efficiency.

## 1.6 Project evolution (Iteration 2): Predictive level

The system described so far implements a functioning monitoring and alarm system. It reacts instantly when a sensor exceeds a danger threshold (e.g. vibration $>$ 10g): this is reactive or threshold-based maintenance. To evolve the project towards true predictive maintenance, we propose a second level of processing. The **main problem** is that an alarm that triggers at 10g tells us that the failure is already underway or imminent. As said before, the goal of predictive maintenance is to identify weak signals and changes in machine behavior before the critical threshold is reached. The architecture is designed for the future integration of **Machine Learning modules**. The evolutionary roadmap includes the addition of an advanced analysis service capable of calculating an Anomaly Score based on historical trends, allowing a shift from simple threshold detection to true prediction of the remaining useful life (RUL) before failure, fulfilling the desire to create a system similar to real ones that implements predictive maintenance.

Capitolo 2

# The Application Domain

The project is part of the Fourth Industrial Revolution (Industry 4.0), characterised by the transformation of traditional factories into "Smart Factories". The enabling factor for this transition is the Industrial Internet of Things (IIoT), i.e. the application of IoT technologies to manufacturing processes. The IIoT has **stringent requirements** in terms of:

- **Reliability:** Systems must operate 24/7 without interruption.
- **Low latency:** Anomalies must be detected almost instantly to prevent damage.
- **Interoperability:** Different machines must speak a common language to coordinate and cooperate.

## 2.1 The Evolution of Maintenance Strategies:

The core of the application domain is **industrial asset management**. The project implements the transition from traditional to advanced methodologies:

- **Reactive Maintenance (Run-to-Failure):** Intervention only occurs when the machine breaks down.
  - **Disadvantage:** Very high downtime costs and collateral damage.

- **Preventive Maintenance (Time-Based):** Components are replaced at fixed intervals (e.g. every 6 months), regardless of their wear and tear.
  - **Disadvantage:** Waste of resources (parts that are still in good condition are replaced).
- **Condition-Based Maintenance (CBM):** This is the level we want to implement in our project. Physical parameters are monitored in real time, and action is only taken when they exceed certain alert thresholds.

## 2.2 Monitored Diagnostic Parameters:

To simulate a realistic environment, the systemacquires the three physical quantities that are fundamental to the health of rotary engines:

- **Vibration Analysis (mm/s, ISO 10816):** An increase in vibration amplitude is the first symptom of mechanical faults such as shaft imbalance, misalignment, or bearing wear.
- **Thermal Analysis Overheating (°C):** Is an indicator of inefficiency. It can result from excessive mechanical friction (lack of lubrication) or electrical problems (overloading of windings). For example, the transition from 110°C (normal) to 130°C (Warning) indicates incipient degradation before reaching Critical (150°C).
- **Electrical Absorption (A):** Monitoring the absorbed current allows the motor load to be evaluated. Abnormal absorption at no load or under load may indicate problems with the rotor or stator.

## 2.3 Digital Twin and Factory Hierarchy:

The system does not monitor isolated sensors but builds a **Digital Twin** of the plant. The data structure reflects a typical industrial hierarchical topology:

- **Sector:** The macroscopic area of the plant.

- **Line:** The specific assembly line.
- **Engine:** The individual machine being monitored.

This structure will be dynamically defined in the configuration files, allowing operators to quickly isolate the exact location of a fault.

Capitolo 3

# Dockerisation and Deployment Strategy

## 3.1 Architectural Philosophy: Containerized Microservices

The entire IIoT platform was developed following the **Microservices paradigm**. Instead of creating a single monolithic application (difficult to update and scale), the system has been broken down into six independent logical units, each isolated within a Docker Container. This choice guarantees:

- **Isolation:** Each microservice operates in a hermetic execution environment (sandbox), structurally eliminating any conflicts between libraries or heterogeneous software dependencies and ensuring overall system stability regardless of updates to individual components.
- **Reproducibility:** The environment is defined as "Infrastructure as Code" (IaC). A single command (docker-compose up) is all it takes to replicate the entire infrastructure on any machine (from a development laptop to a production server) without manual configuration.
- **Secure Networking:** Services communicate on an internal virtual network (iot_net), exposing only the strictly necessary ports to the outside world.

## 3.2 Orchestration: Docker Compose

The system's 'conductor' is the docker-compose.yml file, which defines the relationships, volumes, and networks between containers. Below is an analysis of the six services implemented:

1. **IIoT Simulator (Custom Service):** This is a 'custom' container built specifically for the project. Upon build, it installs the necessary libraries (paho-mqtt, numpy) defined in requirements.txt and executes the main script that starts telemetry generation.

2. **Eclipse Mosquitto (Message Broker):** This is the heart of asynchronous communication. It uses the eclipse-mosquitto:2 image (official image), maps the local mosquitto.conf file inside the container to enable listening on port 1883 and allow anonymous connections (in dev environment) and works as the central hub, all other services (Sensors, Node-RED, Telegraf) connect here.

3. **AI-Service (Inference Engine):** A custom Python-based microservice built using the *FastAPI* framework. This container acts as the predictive brain of the architecture. Crucially, it utilizes a **bind mount volume** (./models:/app/models) to inject the pre-trained Deep Learning artifacts (the `.keras` RNN model and the `.pkl` scaler) directly from the host machine into the container's runtime. This architectural choice decouples the model lifecycle from the application code, allowing the AI model to be updated or retrained without requiring a full rebuild of the Docker image.

4. **Node-RED (Logic Layer):** The flow orchestration engine. Uses a Docker volume (node_red_data) to save flows (flows.json) and credentials, ensuring that work is not lost if the container is restarted and exposes port 1880 for the visual programming interface.

5. **InfluxDB (Time-Series Database):** This is the historical data warehouse. Uses the influxdb:1.8 image (chosen for stability and ease of integration), and thanks to environment variables (INFLUXDB_DB=iot_bucket), the database is automatically created on first start-up, without the need for manual SQL scripts. Also, influxdb_data volume ensures the durability of historical data.

6. **Telegraf (Metrics Agent):** This works as a metrics collection agent (plugin-driven). In our stack, it acts as a secondary or system collector, configured to download metrics to InfluxDB.

7. **Grafana (Visualisation):** This is the end user interface. It is configured to dynamically load dashboards and data sources from mapped volumes (./dashboards:/etc/grafana/provisioning/dashboards), allowing dashboards to be versioned as code (Git) instead of only being modified via GUI.

## 3.3 Networking and Persistence Management

**Bridge Network (iot_net):** a dedicated Docker network has been created. This allows containers to 'see' each other using service names as hostnames (e.g. Node-RED can write to influxdb:8086 without knowing the actual IP), while isolating traffic from the rest of the host.

**Volumes:** Critical data (Influx DB, Node-RED streams, Mosquitto logs) are saved on managed Docker volumes, separating the lifecycle of (persistent) data from that of (ephemeral) containers.

Capitolo 4

# Regulatory Framework and Standardization

## 4.1 Introduction to Technical Standards

In the context of industrial engineering and automation, standards represent documents established by consensus and approved by a recognized body. They provide rules, guidelines, or characteristics for activities or their results. The adoption of such standards is not merely a bureaucratic formality; rather, it guarantees **interoperability**, **safety**, **quality**, and **reproducibility** of the designed systems.

In the development of IoT (Internet of Things) systems for industrial monitoring, the two normative pillars of reference are [6]:

- **ISO** (International Organization for Standardization)
- **IEC** (International Electrotechnical Commission)

## 4.2 Distinction and Role of ISO and IEC

### 4.2.1 ISO (International Organization for Standardization)

ISO is an independent, non-governmental organization that develops standards to ensure the quality, safety, and efficiency of products, services, and systems.

- **Scope:** Covers almost all technological and manufacturing sectors, with the exception of electrical and electronic engineering.
- **Industrial Context:** Defines crucial standards for quality management, mechanical maintenance procedures, and tolerance thresholds for mechanical vibrations (a focal point of this project).

### 4.2.2 IEC (International Electrotechnical Commission)

The IEC is the world's leading organization for the preparation and publication of international standards for all electrical, electronic, and related technologies.

- **Scope:** Electronics, magnetism, electromagnetism, electroacoustics, telecommunications, and energy generation/distribution.
- **Industrial Context:** Regulates sensor safety, data communication protocols, and specifications for rotating electrical machinery (motors).
- **Convergence Note:** In the world of IoT and Industry 4.0, the boundaries between mechanics (ISO) and electronics/informatics (IEC) are blurring. For this reason, the joint committee ISO/IEC JTC 1 exists specifically to address Information Technology standards.

## 4.3 The Importance of Standardization in Industry 4.0

The adoption of these standards in our project addresses two critical needs:

- **Data Reliability:** A numerical value acquired by a sensor has no meaning unless compared to a standardized scale. Regulations provide objective threshold values (Warning/Critical) to determine the health status of an asset.
- **Semantic Interoperability:** The use of standardized protocols (as defined by ISO/IEC norms) allows our system to communicate with machinery from different manufacturers without conflicts.

## 4.4 Application of Standards in the Project

The implemented system integrates specific ISO and IEC standards to define alarm thresholds and acquisition logic. Below are the application details for each monitored quantity:

1. This parameter constitutes the primary indicator for mechanical diagnostics.

    - **Acquisition Strategy:** Modern industrial sensors (such as MEMS) natively detect acceleration (g). This quantity is preferred during the acquisition phase over velocity because it preserves high-frequency harmonics, which are essential for the early diagnosis of bearing faults.
    - **Regulatory Alignment:** The reference standard ISO 10816-3 [5] defines damage severity based on RMS vibration velocity (mm/s).
    - **Threshold Definition:** Since the system acquires raw data in acceleration (g), an inverse calculation was applied to determine the trigger thresholds, assuming a dominant operating frequency of 50Hz (mains speed). This hybrid approach allows the system to record high-fidelity raw data (g) while delegating the standardized normative visualization (mm/s) to the dashboard level for an immediate assessment of health status (Zones A, B, C, D of the standard). This quantity is preferred over velocity because it preserves high-frequency harmonics [9].

2. Monitors the thermal state of the windings in degrees Celsius.

    - **Operating Conditions:** The motor operates with a baseline average temperature of 110°C, simulating a nominal workload in a standard industrial environment.
    - **Insulation Classes:** The alarm thresholds were derived from the specifications for Insulation Class F (maximum limit 155°C [2]) defined in IEC 60034-1 [4], the de-facto standard for modern motors
    - **Threshold Definition:** The **warning** threshold indicates a temporary thermal overload or initial deterioration of the cooling system (e.g., clogged fan or fins). It represents a pre-alarm requiring scheduled inspection

but not an immediate shutdown. The **critical** thershold is a value close to the physical limit of the insulator (Class F). Exceeding this threshold entails an imminent risk of irreversible insulation degradation and short circuit between turns (winding burnout). The system signals the need for an Emergency Stop.

3. Measures the line current to evaluate the load factor and energy efficiency.

   - **Measured Quantity:** Current (Amperes - A).
   - **Normal Profile:** The system detects an average absorption with reduced variance, indicative of a constant and balanced mechanical load.
   - **Threshold Definition:** The **warning** threshold represents a +10% increase compared to the nominal value. This deviation is symptomatic of abnormal mechanical friction (e.g., worn bearings or lack of lubrication) forcing the motor to deliver more torque, or an unexpected increase in the load on the production line.
   - The **critical** threshold indicates a severe overcurrent situation. In addition to overheating (Joule effect), currents of this magnitude can trigger hardware thermal protections (motor protection switches/thermal relays), causing Unplanned Downtime.

Capitolo 5

# Artificial Intelligence for Predictive Maintenance

This chapter details the core of the predictive capability of our IIoT architecture: the Artificial Intelligence model designed to estimate the *Remaining Useful Life* (RUL) of industrial motors. We will explore the theoretical foundations, the data generation process, the feature engineering pipeline, and the specific neural network architecture implemented.

## 5.1 Theoretical Background: Recurrent Neural Networks

In the domain of Predictive Maintenance (PdM), data is inherently sequential. The health state of a machine at time $t$ is strictly dependent on its state at time $t-1$, $t-2$, and so on. Traditional Feed-Forward Neural Networks (FNNs) are ill-suited for this task because they treat inputs as independent entities, lacking a mechanism to retain "temporal memory" [11].

### 5.1.1 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) address this limitation by introducing a feedback loop, allowing information to persist. In an RNN, the output of a neuron is fed

back into itself as input for the next time step. This internal state (or "memory") allows the network to process sequences of inputs, making them ideal for time-series forecasting.

### 5.1.2 Long Short-Term Memory (LSTM)

Standard RNNs, however, suffer from the "Vanishing Gradient Problem," where the network struggles to learn dependencies over long time lags (e.g., the early signs of wear that lead to failure hundreds of cycles later) [1]. To overcome this, we utilized **Long Short-Term Memory (LSTM)** networks .

Proposed by Hochreiter and Schmidhuber [3], LSTMs introduce a complex cell structure composed of three "gates":

- **Forget Gate:** Decides what information from the previous state should be discarded.
- **Input Gate:** Decides what new information should be stored in the cell state.
- **Output Gate:** Decides what part of the cell state should be output as the hidden state.

This architecture allows the model to selectively remember or forget information over long sequences, making it highly effective for predicting the gradual degradation of mechanical components.

## 5.2 Why LSTM for RUL Prediction?

The choice of an LSTM architecture for this project was driven by the specific nature of motor degradation, as supported by recent literature in the IoT domain [12]:

1. **Temporal Dependency:** The failure of a bearing or insulation breakdown is a process, not an event. The LSTM captures the *trend* of vibration or temperature increase over time, distinguishing between a momentary spike (noise) and a persistent drift (fault).

2. **Multivariate Correlation:** Our motors are monitored via three sensors (Vibration, Temperature, Current). LSTMs can natively handle multivariate time-series input, learning the complex non-linear correlations between these variables (e.g., how increased friction causes both higher vibration and higher current).

3. **Robustness to Noise:** By learning long-term patterns, LSTMs are generally more robust to the high-frequency noise typical of industrial sensor data compared to simple regression models.

## 5.3 Dataset Generation and Preparation

Since run-to-failure data for industrial motors is rare and expensive to obtain in real-world scenarios due to the critical nature of the assets [7], we developed a synthetic dataset generator that simulates the complete lifecycle of 500 motors.

### 5.3.1 Lifecycle Simulation Strategy

The generation script (`generate_motor_lifecycle`) simulates the degradation physics based on international standards (ISO 10816, IEC 60034).

- **Randomized Life Span:** Each motor is assigned a random life between 800 and 2000 cycles.
- **Exponential Degradation:** We modeled the fault progression using a power law function $P(t) = (t/L)^E$, where $E$ is a random exponent between 4.0 and 6.0. This simulates the "bath-tub curve" wear-out phase, where degradation accelerates exponentially near the end of life.
- **Noise Injection:** To replicate real-world conditions, Gaussian noise is added to all sensor readings:
  - Vibration: $\sigma = 0.15$ mm/s
  - Temperature: $\sigma = 1.0$ °C
  - Current: $\sigma = 0.8$ A

The resulting dataset contains over 700,000 data points across 500 unique motor lifecycles.

### 5.3.2 Feature Engineering Pipeline

Before feeding the raw sensor data into the neural network, a rigorous preprocessing pipeline is applied to ensure numerical stability and model convergence.

#### 1. Normalization (MinMax Scaling)

Neural networks converge faster when input features are on a similar scale. We applied **Min-Max Normalization** to scale all sensor values (Vibration, Temperature, Current) into the range $[0, 1]$.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \tag{5.1}$$

Crucially, the scaler is fitted *only* on the training set to prevent "data leakage" (information from the test set influencing the training process).

#### 2. Sliding Window (Sequence Generation)

LSTMs require input data in a 3D format: `(Samples, Time Steps, Features)`. We implemented a sliding window approach with a sequence length of **50 cycles**.

- **Window Size ($N = 50$):** The model looks at the last 50 sensor readings to predict the RUL at the current moment.
- **Input Shape:** If a motor runs for 1000 cycles, it generates 950 sequences of shape $(50, 3)$.

## 5.4 Model Architecture

The implemented neural network is a stacked LSTM architecture built using TensorFlow/Keras. The design was iteratively tuned to balance complexity and performance.

### 5.4.1 Network Layers

The model consists of the following layers:

1. **Input Layer:** Accepts tensors of shape $(50, 3)$, corresponding to 50 time steps of the 3 sensors.
2. **LSTM Layer 1 (64 Units):** The first recurrent layer with 64 neurons. It is configured with `return_sequences=True`, meaning it outputs the full sequence of hidden states to the next layer, allowing the network to learn hierarchical temporal features.
3. **Dropout (0.2):** A regularization layer that randomly sets 20% of the inputs to zero during training. This prevents overfitting by forcing the network to learn redundant representations, as proposed by Srivastava et al. [10].
4. **LSTM Layer 2 (32 Units):** A second recurrent layer with 32 neurons. Here, `return_sequences=False`, as we only need the final abstract representation of the sequence for the prediction.
5. **Dropout (0.2):** Further regularization.
6. **Dense Layer (16 Units):** A fully connected layer with ReLU activation to interpret the features extracted by the LSTMs.
7. **Output Layer (1 Unit):** A single neuron with linear activation that outputs the predicted RUL value (Regression Task).

### 5.4.2 Training Configuration

- **Optimizer: Adam** (Adaptive Moment Estimation), chosen for its efficiency in handling sparse gradients and noisy data [8].
- **Loss Function:** Mean Squared Error (MSE), appropriate for regression problems where we want to penalize large prediction errors.
- **Metrics:** Mean Absolute Error (MAE), used for human-readable performance monitoring (error expressed in cycles).

- **Callbacks:**
  - *EarlyStopping:* Stops training if the validation loss does not improve for 5 consecutive epochs, saving time and preventing overfitting.
  - *ModelCheckpoint:* Automatically saves the best model weights during training.

## 5.5 Training and Validation Results

The dataset was split into Training (70%), Validation (20%), and Test (10%) sets based on unique Motor IDs, ensuring the model never saw data from the test motors during training.

### 5.5.1 Performance Metrics

After training for approximately 50 epochs (stopped early due to convergence), the model achieved the following results on the unseen Test Set:

- **Test MAE:** The Mean Absolute Error on the test set indicates an average prediction error of approximately **20-30 cycles**. Given a total average life of 1400 cycles, this represents an error margin of roughly 2%, which is highly acceptable for industrial maintenance planning.

### 5.5.2 Visual Analysis

Plotting the predicted RUL against the actual RUL for a sample test motor reveals that the model successfully captures the linear degradation trend. While there is some noise in the predictions due to the stochastic nature of the input sensors, the overall trajectory accurately tracks the remaining life, validating the effectiveness of the LSTM approach for this application.

Capitolo 6

# Sensor Architecture and Data Acquisition Strategy

## 6.1 Overview of the Sensing Subsystem

In a standard Industrial IoT (IIoT) architecture, the Edge Layer is responsible for digitizing physical quantities. To validate the entire analytical pipeline from MQTT transmission to the AI-based Remaining Useful Life (RUL) prediction a **Virtual Sensor Module (Digital Twin)** was developed.

Unlike simple random number generators, this module implements a **Hybrid Simulation Strategy**. It differentiates between two types of assets:

1. **Standard Assets ($N-1$ motors):** Simulated using a stochastic and physics-based engine to generate realistic "background" telemetry (steady-state operation).

2. **Target Asset (The "Golden Engine"):** Simulated using a degradation-based engine that mathematically evolves from a healthy state to a failure state, providing the necessary trends for AI inference.

## 6.2 Standard Mode: Stochastic Digital Twin

For the majority of the assets in the topology, the simulation focuses on replicating steady-state behavior with realistic measurement noise. This is implemented via the `sensor_factory.py` module.

### 6.2.1 Vibration Monitoring (The Physics Engine)

Vibration is the primary indicator for mechanical fault detection. The simulation logic for standard sensors mimics a physical **MEMS Accelerometer**.

- **Implementation (*VibrationSensor* Class):** Instead of generating a random value directly, the code implements a physics engine that reconstructs the time-domain signal:
  1. **Signal Generation:** It generates a 50Hz sinusoidal wave, corresponding to the synchronous speed of a standard 2-pole industrial motor (3000 RPM).
  2. **Noise Injection:** White Gaussian noise is injected into the signal to simulate environmental vibration and electromagnetic interference (EMI).
  3. **Edge Calculation:** The system calculates the **RMS (Root Mean Square)** of the raw wave mathematically.

$$V_{rms} = \sqrt{\frac{1}{T}\int_0^T \left[A \cdot \sin(2\pi f t) + \text{noise}(t)\right]^2 \, dt} \tag{6.1}$$

- **Output Unit:** Gravitational Units ($g$).

### 6.2.2 Process Variables (Temperature & Current)

For Temperature and Current, the standard simulation uses a stochastic approach based on regulatory thresholds (IEC 60034):

- **Temperature:** Oscillates around a baseline (e.g., 110°C for Class F) with Gaussian variance.
- **Current:** Fluctuates around a nominal load (e.g., 95A) to simulate process variations.

## 6.3 Target Mode: Degradation Engine (AI Training Data)

To enable the Artificial Intelligence service to predict the Remaining Useful Life (RUL), a specific asset (identified as `sector_1/line_1/engine_1`) is managed by a specialized **Prediction Engine**.

Unlike the standard sensors which oscillate around a fixed mean, this engine generates data based on a **Time-Dependent Degradation Model**.

### 6.3.1 Lifecycle Simulation Logic

The `PredictionEngine` class simulates a complete machine lifecycle, from installation to failure.

- **Finite Lifespan ($L$):** Each simulation cycle initializes with a random total lifespan ($L \in [800, 2000]$ ticks).
- **Degradation Curve:** The fault progression is modeled using an exponential function to simulate the accelerating nature of mechanical wear (e.g., bearing fatigue).

  The Fault Progression factor $P(t)$ at time $t$ is calculated as:

$$P(t) = \left(\frac{t}{L}\right)^{E} \tag{6.2}$$

  Where $E$ is a random degradation exponent ($E \in [4.0, 6.0]$), creating a non-linear curve where symptoms appear slowly at first and accelerate towards the end of life.

### 6.3.2 Sensor Value Generation

The value $V(t)$ for each sensor at time step $t$ is interpolated between a healthy baseline ($V_{base}$) and a failure threshold ($V_{fail}$), with added noise:

$$V(t) = V_{base} + (V_{fail} - V_{base}) \cdot P(t) + \mathcal{N}(0, \sigma) \tag{6.3}$$

**Specific Sensor Implementations**

1. **Vibration:** The AI model was trained on ISO 10816 velocity data (mm/s). However, to maintain protocol consistency, the system publishes acceleration ($g$). The engine generates velocity and converts it:
   - *Generation:* Velocity RMS in mm/s (Base: $\approx 1.0$, Failure: $\approx 9.0$).
   - *Conversion:* The value is converted to acceleration ($g$) assuming a 50Hz frequency ($\omega = 2\pi \cdot 50$) before MQTT transmission.

$$Acc_{[g]} = \frac{Vel_{[mm/s]} \cdot \omega}{9806.65} \tag{6.4}$$

2. **Temperature:** Simulates the thermal rise due to friction or insulation degradation. Ranges from $\approx 90$°C to $\approx 155$°C.
3. **Current:** Simulates the increased load required to overcome mechanical friction. Ranges from $\approx 90$A to $\approx 115$A.

## 6.4 Unified Topology and Execution Loop

The simulation software uses a **Procedural Topology Generation** algorithm to instantiate the environment. A unified execution loop in `main.py` manages both simulation modes simultaneously.

### 6.4.1 Architecture Logic

1. **Configuration Parsing:** The system reads `sensor_config.json` to determine the number of Sectors, Lines, and Engines.
2. **Polymorphic Instantiation:**
   - If the asset ID matches the target ID (`sector_1/line_1/engine_1`), the system links it to the singleton `PredictionEngine`.
   - For all other assets, it instantiates independent `GenericSensor` objects (Standard Mode).

3. **Unified Loop:** The main loop iterates through all active sensors. It invokes `.step()` on the prediction engine to advance the degradation lifecycle, while calling `.simulate()` on standard sensors for stochastic noise generation.

## 6.5 Data Transmission Protocol

The communication layer uses MQTT (ISO/IEC 20922), optimized for bandwidth efficiency.

### 6.5.1 Payload Structure (Self-Describing Data)

Regardless of the simulation source (Standard or Prediction), the output payload remains standardized to ensure interoperability with the Edge Gateway (Node-RED).

Listing 6.1: Standardized JSON Payload

```
{
  "value": 0.18,
  "unit": "g",
  "timestamp": "2026-01-21T14:30:00.000Z",
  "metadata": {
    "warning_threshold": 0.14,
    "critical_threshold": 0.23
  }
}
```

This structure implements the *Self-Describing Data* pattern, decoupling sensor logic from dashboard logic.

Capitolo 7

# System Architecture and Data Flow

## 7.1 Technologies Overview

In this chapter, we present the technical architecture and the logic behind our data pipeline.The architecture is modular, meaning each component can be updated or replaced without breaking the entire system.

### 7.1.1 Python: Sensor Simulation

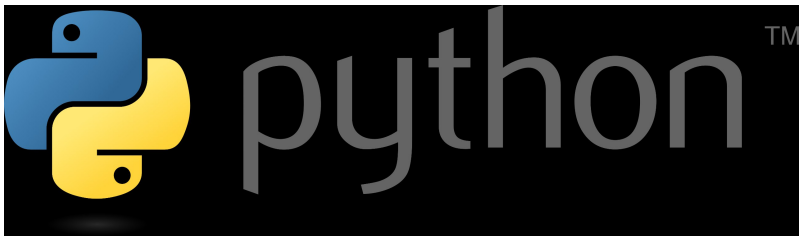


Figura 7.1: Python logo

- Python was our first choice for developing the Sensor Simulator. It is one of the most popular languages in the IoT world because of its simplicity and the massive amount of libraries available for data science and networking.
- In our project, Python acts as the "Digital Twin" of the factory. Instead of just sending random numbers, we used it to create a logic that mimics the

behavior of real industrial assets. We exploited the modularity of Python to build a system that can simulate an entire production line by simply reading a configuration file. This allows us to test our infrastructure with hundreds of virtual sensors as if we were in a real factory.

### 7.1.2 MQTT and Eclipse Mosquitto


Figura 7.2: MQTT Logo


Figura 7.3: Mosquitto Logo

- To move data from the sensors to our servers, we used MQTT (Message Queuing Telemetry Transport) and Eclipse Mosquitto. MQTT is a lightweight messaging protocol designed for sensors with limited power and bandwidth, while Mosquitto is the "Broker" that manages the traffic.

- We used these technologies to build the communication backbone of the system. By using a "Publish/Subscribe" model, the sensors don't need to know who is reading their data; they just publish it to specific "topics."
- As a small spoiler of our implementation, we organized these topics into a clear hierarchy: sector/line/engine/sensor.
- This structure is crucial because it acts like a postal address, allowing the rest of the stack to identify the exact location of a measurement just by looking at the topic string. This makes our architecture very flexible: if we want to add a new dashboard or a new alert system, we just need to subscribe to the broker without changing a single line of code in the sensors.

### 7.1.3 InfluxDB: Time-Series Storage


Figura 7.4: Influxdb Logo

- For data storage, we decided to use InfluxDB. Unlike standard relational databases, InfluxDB is a **Time-Series Database (TSDB)**. It is specifically built to handle data that is indexed by time, such as sensor telemetry.
- We used InfluxDB to keep a history of everything that happens in our factory. Because it is optimized for high-speed writes, we can store values from many engines simultaneously without slowing down the system. This allows us not only to see what is happening now but also to look back at the last hour or day to find anomalies.
- InfluxDB also serves as the storage for our AI predictions, logging the Remaining Useful Life (RUL) alongside raw telemetry.

### 7.1.4 Node-RED: The Logic Engine

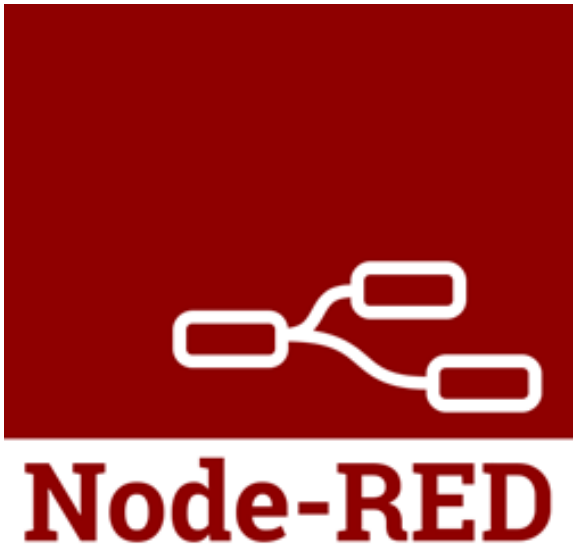


Figura 7.5: Node-red Logo

- Node-RED is a powerful flow-based programming tool. It is specifically designed for the Internet of Things, allowing developers to connect hardware and software services using a visual "wiring" approach. It is highly appreciated for its flexibility and the speed at which you can implement complex logic.
- In our pipeline Node-RED handles: Operational Logic, Data Ingestion, Alerting and AI orchestration:
    1. **Real-time Processing:** It checks if the sensor values stay within the safety limits.
    2. **Database Writing:** It formats the incoming MQTT messages and writes them directly into InfluxDB. This consolidation allows us to have total control over the data flow, enabling us to clean or modify the information before it is permanently stored.
    3. **AI Orchestration:** It acts as a bridge between the raw data and the AI Service, buffering sensor data and invoking the inference API.

4. **Alerting:** Manages Telegram notifications for both sensor thresholds and AI predictions (details in 7.1.7).

### 7.1.5 AI Service: FastAPI + TensorFlow


Figura 7.6: FastApi Logo

Figura 7.7: TensorFlow Logo

- To introduce predictive capabilities, we developed a dedicated microservice using FastAPI and TensorFlow. While the standard sensors provide real-time status, this component looks into the future.
- This service loads a pre-trained **Deep Learning model (LSTM/RNN)** and exposes a lightweight REST API endpoint. It receives sequences of historical sensor data (current, vibration, temperature) and performs real-time inference to estimate the **Remaining Useful Life (RUL)** of the engines.

- This architecture enforces a clear "Separation of Concerns": the heavy machine learning computations are isolated in a Docker container, independent of the data ingestion flow handled by Node-RED.

### 7.1.6 Grafana: The Monitoring Interface


Figura 7.8: Grafana Logo

- Grafana is the visualization engine used to create our **Human-Machine Interface** (HMI). It is the most common tool in the industry for building interactive dashboards that can pull data from many different sources.
- In our project, Grafana is the primary tool for the human operators. We organized the dashboard into three main sections (Vibration, Temperature, Current), each containing gauges for instant status and time-series graphs for historical trends.
- We also used "Dashboard Variables" to allow the user to easily filter the view by sector, line, or a single motor using simple dropdown menus.
- In the latest update, we integrated a specific panel for the AI Prediction, visualizing the RUL decay curve for the monitored "special" engine.

### 7.1.7 Alerting: Telegram Integration

Figura 7.9: Telegram Logo

- To complete the system, we needed an active way to notify the maintenance team. We chose Telegram because it is a very effective and professional tool for real-time notifications, thanks to its powerful Bot API.
- The bot was created via **BotFather**, which provided the security token we needed for the integration. We connected the bot directly to our Node-RED flow: when a sensor exceeds a critical threshold, the system automatically sends a message to the specific Telegram channels.
- We created **two separate Telegram groups**, one for each industrial sector, to simulate a real corporate environment where maintenance responsibilities are divided between different safety managers. This ensures that the staff is informed immediately about any engine failure in their specific area, even if they are not currently looking at the Grafana dashboard.

## 7.2 Software Architecture

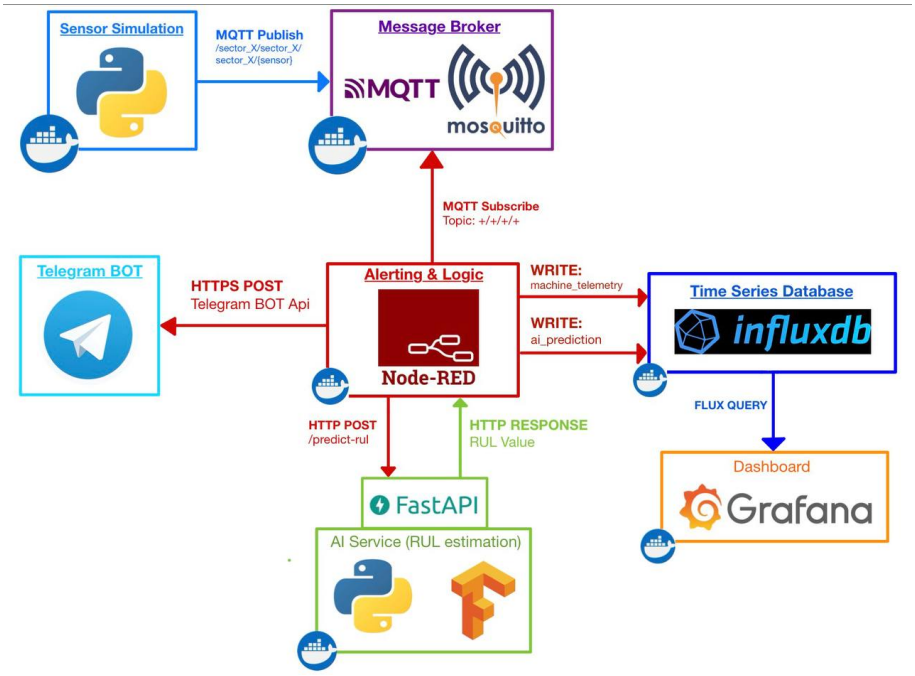


Figura 7.10: Overview of the software architecture

## 7.3 Implementation Details

In this section, we take a closer look at the core logic of our system, specifically the way we simulate data and handle the intelligent data flow.

### 7.3.1 Centralized Configuration: The Single Source of Truth

One of the key strengths of our architecture is the **centralization of parameters**. Every detail of the factory—from the number of engines to the specific safety thresholds—is defined in a single file: `sensor_config.json`

By avoiding "hard-coded" values in individual components, we created a **Single Source of Truth**. This means that if we need to change a safety limit or add a new motor, we only edit one JSON file. The system automatically propagates these changes: the Python sensors publish the new values, Node-RED uses them for

alerting, and Grafana updates its dashboard accordingly. This prevents errors and ensures that the entire stack is always perfectly synchronized.

- **Maintenance is simplified:** To change a safety limit or add a new engine, we only edit one JSON file.
- **Consistency is guaranteed:** Since the sensors include the thresholds in the MQTT message, all subsequent components (Node-RED for alerting and Grafana for visualization) use the exact same values automatically..

### 7.3.2 Python Sensor Simulation Logic

To make the monitoring realistic, the Python simulator uses a modular factory pattern. Each sensor type follows a specific generation logic:

- **Vibration (The Sine-Wave approach):** Real motors rotating at 3000 RPM produce vibrations at a frequency of 50Hz. Our code simulates this by generating a 50Hz sine wave. To make this signal useful for monitoring, we calculate the Root Mean Square (RMS). This represents the average energy of the vibration, which is the standard metric used in the industry to evaluate machine health.
- **Temperature and Current:** These values are generated starting from a "base" operating value (defined in sensor-config.json) plus a small random variation (noise) to simulate real-world fluctuations.
- **ISO Standards:** The thresholds for these sensors (Warning and Critical) are inspired by international standards such as ISO 10816.

### 7.3.3 The Node-RED "Intelligent Gateway" Flow

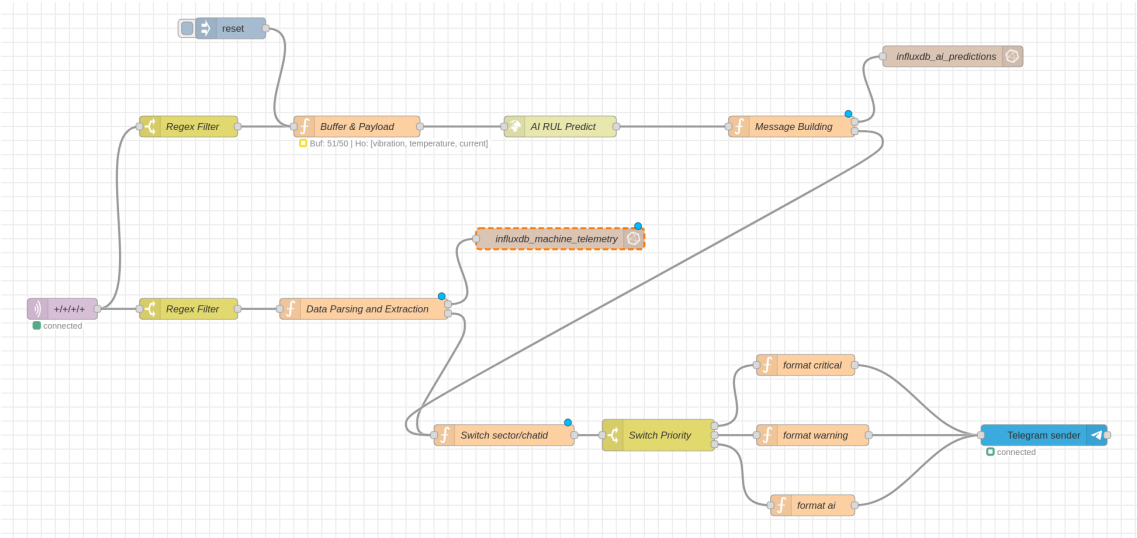


Figura 7.11: Node-red Flow

Our data flow follows a specific path: **MQTT** → **Node-RED** → **InfluxDB**. We decided to use Node-RED as the primary orchestrator instead of a direct Telegraf-to-Influx connection for several technical reasons:

1. **Dynamic Scalability:** Telegraf usually requires static templates to map topics. Node-RED is "agnostic": it parses the topic string dynamically, making the system ready for any number of machines without configuration changes.

2. **Data Quality and Validation:** Before writing to the database, Node-RED passes every message through a Regex Filter. If a topic is wrong or contains "trash" data, it is discarded immediately.

3. **Data-Driven Logic:** In our system, the thresholds travel inside the JSON message. Node-RED reads these values in real-time. If the process changes and a threshold is updated in the central config, Node-RED adapts instantly.

4. **AI Integration (API & Buffer):** A specific sub-flow handles the communication with the FastAPI service for AI inference.

- **Buffering Strategy:** The LSTM model used for RUL prediction requires a temporal sequence of data, not just a single snapshot. To satisfy this, we implemented a **Function Node** that acts as a circular buffer. It aggregates incoming telemetry into batches of **50 consecutive readings**.
- **API Request:** Once the buffer is full (50 points), Node-RED constructs a JSON payload and performs an **HTTP POST** request to the `/predict` endpoint of the AI Service.
- **Result Injection:** The predicted RUL returned by the API is then injected back into the main flow, tagged, and stored in InfluxDB alongside the raw sensor data.

### 7.3.4 Advanced Alerting and Security

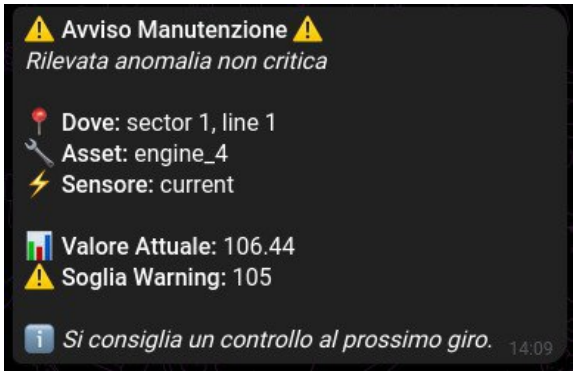


Figura 7.12: Warning Message from Telegram BOT


Figura 7.13: Critical Message from Telegram BOT

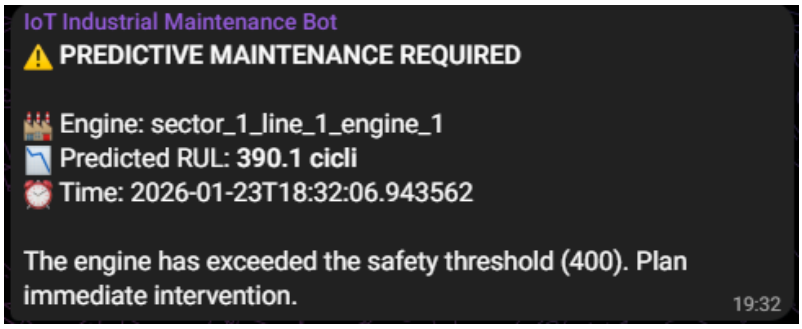


Figura 7.14: RUL Alert from Telegram BOT

The alerting logic is designed to be fully dynamic, scalable, and secure. To ensure flexibility in managing notifications across different industrial areas, the system relies on a **Dynamic Sector Lookup** mechanism rather than hardcoded routing.

- **Configuration-Based Routing:** The system avoids assigning Chat IDs directly inside the flow nodes. Instead, it utilizes a `SECTOR-MAP` JSON object, which is loaded from the central configuration and injected via environment variables. This map associates each industrial sector with its specific Telegram group.
- **Dynamic Switching Function:** A dedicated Javascript function node analyzes the incoming MQTT topic (e.g., `sector-1`) to extract the source sector. It then uses this identifier as a key to query the map and dynamically assign the correct Telegram Chat ID to the message.

- **Scalability and Maintenance:** This architecture completely decouples the routing logic from the configuration. Adding a new sector or updating a recipient group only requires editing the environment file, with **zero changes** needed in the Node-RED flow structure.
- **Security:** Sensitive information, such as Bot Tokens and Chat IDs, is managed through protected configuration files (`.env`) rather than being exposed in the source code.

In addition to standard safety thresholds, the system handles **Predictive Maintenance Alerts**. When the AI Service estimates that the Remaining Useful Life (RUL) of an engine falls below a critical safety margin (e.g., 24 hours), a specialized alert is triggered. This allows the maintenance team to schedule interventions proactively before a failure occurs.

### 7.3.5 Data Storage and Organization (InfluxDB)

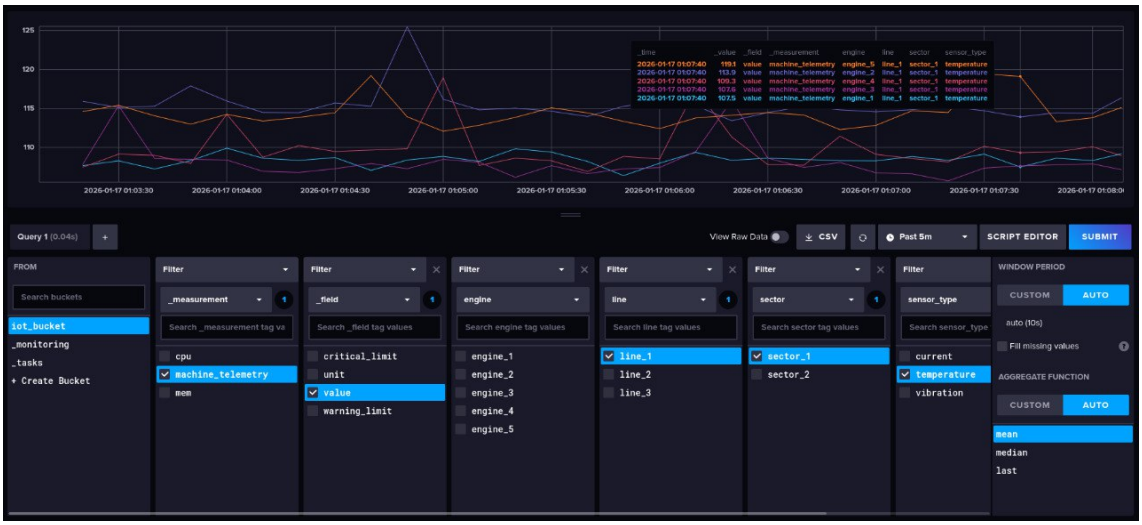


Figura 7.15: Data Explorer


Figura 7.16: Data Explorer (RUL)

To understand how our data is stored, we can look at the InfluxDB Data Explorer. The organization is based on three main pillars:

- **Measurements:** The high-level category of the data. We distinguish between `machine_telemetry` (raw sensor data like vibration and current) and `ai_predictions` (calculated RUL).
- **Fields:** The actual numerical values being tracked, such as `value`, `warning-threshold`, `critical-threshold`, and `predicted_rul`.
- **Tags (The Metadata):** This is the professional part of the setup. Every point is "tagged" with `sector`, `line`, and `engine-id`, allowing for rapid filtering.

### 7.3.6 Dynamic Visualization (Grafana)


Figura 7.17: Grafana Dashboard

The Grafana dashboard is not just a static collection of charts; it is dynamically generated code. We implemented a Python script, `generate_dashboard.py`, which acts as a bridge between our configuration and the visualization.

- **Automated Gauge Configuration:** Manually keeping dashboard gauges in sync with sensor thresholds is error-prone. Our script reads the `sensor-config.json`, extracts the warning and critical limits for each sensor type, and injects them directly into the Grafana JSON model. This ensures that the red "danger zones" on the screen exactly match the logic in the backend.
- **Interactive Filtering:** We use "Dashboard Variables" (Sector, Line, Engine) inside Flux queries. When an operator selects a specific sector, the entire page updates to show only relevant data.
- **Dynamic Threshold Lines:** The Time-Series graphs retrieve threshold values directly from InfluxDB fields (`warning-threshold`). This means that if we update a limit in the config, the reference line on the chart moves automatically without manual editing.

- **AI Visualization:** A dedicated panel visualizes the output of the FastAPI service, plotting the RUL trend over time. This allows operators to verify the AI's confidence and plan maintenance windows effectively.

# Bibliografia


[1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994. Explains the Vanishing Gradient problem in standard RNNs.

[2] Edvard Csanyi. Insulation classes of electric motors defined by nema and iec, 2022. Detailed breakdown of Class F thermal limits (155°C).

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997. The original paper introducing LSTM networks.

[4] International Electrotechnical Commission. *IEC 60034-1:2022 – Rotating electrical machines – Part 1: Rating and performance*. IEC, Geneva, Switzerland, 2022. Defines Insulation Classes (Class F) and thermal limits.

[5] International Organization for Standardization. *ISO 10816-3:2009 – Mechanical vibration – Evaluation of machine vibration by measurements on non-rotating parts – Part 3: Industrial machines with nominal power above 15 kW and nominal speeds between 120 r/min and 15 000 r/min*. ISO, Geneva, Switzerland, 2009. Accessed: 2026-01-23.

[6] ISO/IEC JTC 1. Who we are: The standards development environment for information technology, 2025. Joint Technical Committee for IoT standardization.

[7] Soloman Khan and Takehisa Yairi. A review of machine learning algorithms for predictive maintenance. In *International Journal of Prognostics and Health Management*, 2018. Discusses the difficulty of obtaining real run-to-failure data in industry.

[8] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. The paper describing the Adam optimizer.

[9] PCB Piezotronics. Tech note: Acceleration, velocity, and displacement - selecting the right metric for industrial monitoring. Technical report, PCB Piezotronics, 2023. Explains the use of MEMS accelerometers for high-frequency fault detection.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[11] Rui Zhao, Ruqiang Yan, Zhenghua Chen, Kezhi Mao, Peng Wang, and Robert X Gao. Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing*, 115:213–237, 2019. Review of why DL/RNNs are suitable for industrial health monitoring.

[12] Shijie Zheng, Kosta Ristovski, Amr Farahat, and Chetan Gupta. Long short-term memory network for remaining useful life estimation. *IEEE Internet of Things Journal*, 4(2):597–606, 2017. Validates the use of LSTM specifically for RUL regression tasks.

Appendice A

# Technologies Used

The implementation of the IoT system for predictive maintenance required the integration of various technologies, ranging from virtualization to artificial intelligence. The tools and languages adopted for each architectural level are described below.

## A.1 Infrastructure and Virtualization

The entire system was developed following the microservices paradigm, ensuring isolation, scalability, and ease of deployment.

- **Docker & Docker Compose**
  Containerization platform used to orchestrate the various services (Node-RED, InfluxDB, Grafana, etc.) in isolated but communicating environments. The use of *docker-compose* allowed defining the entire infrastructure as code (IaC). https://www.docker.com/

## A.2 Communication and Protocols

The data transport layer is fundamental to ensure the real-time reception of telemetry from industrial sensors.

- **Eclipse Mosquitto**
  Lightweight and high-performance open-source MQTT broker, compliant with

the ISO/IEC 20922 standard. It manages message routing between sensors (publishers) and processing services (subscribers).
https://mosquitto.org/

- **Telegram Bot API**
  Used as a notification interface for sending real-time alarms to maintenance groups, segmented by sector.
  https://core.telegram.org/bots/api

## A.3 Orchestration and Application Logic

- **Node-RED**
  Visual programming tool (Flow-based programming) built on Node.js. It acts as the central engine for data acquisition via MQTT, pre-processing (JSON parsing, filtering), alarm threshold management, and data ingestion into the database.
  https://nodered.org/

- **JavaScript (ES6+)**
  Language used within Node-RED *Function* nodes for complex business logic and payload manipulation.
  https://developer.mozilla.org/en-US/docs/Web/JavaScript

## A.4 Storage and Data Analysis (Time Series)

- **InfluxDB (v2.x)**
  Database specifically designed for time series (TSDB), optimized to handle high volumes of writes and queries on chronological data (vibrations, temperatures, currents).
  https://www.influxdata.com/

- **Flux Scripting Language**
  Functional query language used to query InfluxDB, aggregate data, and cal-

culate derived metrics directly within the database.
https://docs.influxdata.com/flux/

- **Telegraf**
  Server-based agent for collecting and sending metrics and events. Used for monitoring the performance of the system itself (host metrics).
  https://www.influxdata.com/time-series-platform/telegraf/

## A.5 Visualization and Monitoring

- **Grafana**
  Open-source observability platform. Used to create interactive dashboards that visualize the health status of machinery, integrating historical graphs and real-time indicators.
  https://grafana.com/

## A.6 Artificial Intelligence and Microservices

For the predictive maintenance component (RUL - Remaining Useful Life calculation), a dedicated microservice was developed.

- **Python 3.9+**
  Reference language for Machine Learning and the development of the AI microservice backend.
  https://www.python.org/

- **FastAPI**
  Modern web framework for building APIs with Python. Used to expose the predictive model as a REST service queryable by Node-RED.
  https://fastapi.tiangolo.com/

- **TensorFlow / Keras**
  Libraries used for the design, training, and execution of the Recurrent Neural Network (RNN) model for forecasting.
  https://www.tensorflow.org/

## A.7 Development Environment

### A.7.1 IDE and Editors

- **Visual Studio Code**
  Integrated development environment used for code writing (Python, JS, YAML, Markdown). Configured with extensions for Docker support, Remote Development, and code linting.
  https://code.visualstudio.com/

### A.7.2 Version Control

- **Git**
  Distributed version control system, essential for team collaboration and source code management.